

Repaso de RAG: recuperar, aumentar, generar

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 11 de 12 · Notas en español (es-ES)

La sesión 11 (26 de agosto de 2026) empezó por la carpeta PDF del grupo y recapituló la **generación aumentada por recuperación**. La sesión 10 ya había planteado el PDF de 1.000 páginas: buscar un correo del autor metiendo el libro entero en Claude malgasta tokens y, pasado un tamaño, es imposible en una sola llamada. RAG es el patrón que trae *solo las rebanadas útiles*.

Las tres letras, sin misticismo

1. **Recuperar** — de un corpus (PDF, sitios, bases de datos, incluso transcripciones) encontrar los pocos fragmentos que encajan con la pregunta.
2. **Aumentar** — añadir esos textos al prompt (pregunta + evidencia).
3. **Generar** — Claude escribe la respuesta a partir de ese contexto más pequeño y pertinente — no a partir de 10.000 páginas mudas.

El LLM sigue generando. RAG no sustituye a Claude; cambia *por qué tokens pagáis a la entrada*.

LA DOCUMENTACIÓN SE MUEVE; EL PATRÓN NO

Los proveedores vectoriales de aula y las APIs de cuaderno cambian. Conservad la tubería (cargar → partir → embeber → guardar → buscar → aumentar → generar) y verificad el comportamiento vigente de PDF/archivos en **PDF support** y citas si el examen pregunta por respuestas fundamentadas.

Preparación fuera de línea frente a consulta en línea

No embebeis la biblioteca en cada pregunta. La preparación va primero:

- **Cargar** documentos a texto.
- **Partir / trocear** en piezas que aún signifiquen algo para el negocio (el tamaño es una decisión de caso de uso, no una constante universal).
- **Solapar** un poco (caracteres o frases) cuando necesitáis continuidad al corte — típico en manuales.
- **Embeber** cada trozo con un *modelo de embeddings* (texto entra, vector sale — no un LLM, que es texto entra, texto sale).
- **Guardar** vectores y el texto original del trozo en un almacén vectorial para que la búsqueda devuelva prosa, no solo números.

En el momento de la pregunta: embebed el **prompt**, ejecutad búsqueda por similitud (coseno era el valor por defecto del laboratorio; se nombraron MMR y BM25 según la base de datos), tomad los k mejores (dos aciertos de series de Fourier en el relato en lugar de 175), aumentad, generad.

El troceado es una decisión de producto

El Markdown de demostración (un conjunto de estilo investigación médica, no una guía de estilo del examen) se partió por patrones de encabezado (##) con una regex. Lalit preguntó si valía un parser Markdown; sí — la regex era pedagogía. Otros cortes: recuento de caracteres, frases, estructura del documento (encabezados frente a bloques de código en .md). Las longitudes iban de decenas de caracteres a casi dos mil: evidencia de que «un único tamaño de trozo» es una trampa.

Búsqueda semántica, no igualdad de cadenas

No encontraréis la frase del usuario literal en un trozo. Los embeddings colocan significados próximos. La similitud del coseno (o el equivalente del proveedor) puntúa ese vecindario. Los umbrales son **valores por defecto del proveedor y del método** que podéis anular; Vijay rechazó un número mágico para todas las bases. Ajustad recuerdo frente a ruido según el corpus.

Metadatos frente a un único pozo

Ejemplo universitario: los PDF de matemáticas, ciencias e IA comparten conceptos (series de Fourier en más de una asignatura). Podéis etiquetar cada trozo con metadatos de fichero y filtrar al consultar, o volcar 100+50+25 trozos en un pozo y buscarlo todo. Las bases vectoriales compartidas entre casos de uso valen; el riesgo de alucinación sube si recuperáis la política de otra facultad. La intención también puede ser una *herramienta* que elige qué colección consultar: RAG a menudo se envuelve como herramienta.

Cómo se sienta RAG junto a herramientas y MCP

Una función de recuperación es una herramienta ordinaria: el argumento es la pregunta (o su embedding), el resultado es texto de trozos. Puede vivir en proceso o detrás de MCP. Los agentes (capítulo 03) orquestan «cuándo recuperar». Las plataformas no-code (knowledge bases de Bedrock + auto-sync de S3, OpenSearch) implementan las mismas tres letras con consolas en lugar de cuadernos. La entrega centrada en Claude de este curso se quedó en la **vía de código**.

Coste y caché

Cada golpe que vuelve a embeber y buscar cuesta latencia y dinero. Lalit preguntó por caché: sí, en aplicación, base de datos o red (se nombró Redis). Es diseño de servicio, no un rasgo solo de Claude. La caché de prompts (módulo 09) aún puede aplicar al prompt *umentado* si el prefijo es estable: comprobad la documentación vigente.

Documentación oficial

- **Visión general de ingeniería de prompts**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Cómo escribir el prompt aumentado tras la recuperación.

- **Soporte de PDF**

<https://docs.claude.com/en/docs/build-with-claude/pdf-support>

Cuándo enviáis archivos directamente frente a recuperar trozos.

- **Citas**

<https://docs.claude.com/en/docs/build-with-claude/citations>

Respuestas fundamentadas a partir de documentos aportados.

- **Caché de prompts**

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

Si el prefijo RAG se repite entre llamadas.

- **Visión general de tool use**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

La recuperación envuelta como herramienta.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.